

Shipay — Desafio Backend

Questão 2 — Relatório de utilização do serviço de lançamentos

Proposta de arquitetura para gerar um relatório de utilização por cliente em um intervalo de 30 dias, minimizando o impacto no fluxo principal da API de lançamento de foguetes.

1. Objetivo e premissas

O endpoint de lançamento é o principal produto da empresa e, portanto, deve permanecer como o fluxo crítico. A solução proposta reutiliza as informações já presentes na solicitação existente, sem exigir uma nova chamada do cliente para alimentar o relatório.

Os dados necessários ao reporting são encaminhados para um workflow separado e armazenados em uma base de dados paralela à base do serviço de lançamentos. Uma falha no workflow de reporting não deve impedir nem alterar o resultado de um lançamento já concluído.

2. Arquitetura proposta

Após a conclusão bem-sucedida do lançamento, a aplicação cria um evento `RocketLaunchedEvent`. Um `ReportingEventPublisher`, novo componente proposto para a solução, publica esse evento em uma fila SQS. Um `Reporting Consumer` independente consome as mensagens e persiste os dados no `Reporting DB`.

Componente	Responsabilidade
Launch API / LaunchService	Executar o lançamento e produzir o resultado retornado ao cliente.
ReportingEventPublisher	Abstrair a publicação dos eventos de reporting no mecanismo de mensageria.
RocketLaunchedEvent	Representar o lançamento concluído com os dados mínimos necessários ao reporting.
SQS — Reporting Queue	Desacoplar o workflow secundário do processamento principal e absorver falhas temporárias.
Reporting Consumer	Consumir, persistir e confirmar mensagens somente após a persistência bem-sucedida.
Reporting DB	Manter a projeção dos lançamentos usada nas consultas de reporting.

3. Fluxo principal

O fluxo pode ser resumido como: Cliente → POST `/v1/rocket/launch` → Launch API → LaunchService → lançamento concluído. A partir desse ponto, existem dois caminhos: o resultado segue para a resposta HTTP e o evento segue para o workflow de reporting.

O `Reporting DB` não participa da conclusão do lançamento. O serviço principal também não precisa consultar o `Reporting DB` para responder à solicitação.

4. Reutilização da solicitação existente

A premissa de utilizar as solicitações/requests em comum com o serviço atual significa, nesta proposta, aproveitar o mesmo request recebido pelo endpoint de lançamento. O `customer_id` já existente em `LaunchRequestBody` é utilizado na construção do evento. Não é criado um novo endpoint ou uma segunda solicitação do cliente para o reporting.

5. Alteração proposta no endpoint existente

Antes:

```
return await service.launch(trace_id=trace_id,
customer_id=schema.customer_id,
countdown_status=countdown_status,
pre_flight=pre_flight)
```

Depois:

```

async def launch(...,
reporting_event_publisher: ReportingEventPublisher = Depends(
Provide[Container.reporting_event_publisher]
)):
...

launch_result = await service.launch(
trace_id=trace_id,
customer_id=schema.customer_id,
countdown_status=countdown_status,
pre_flight=pre_flight
)

event_id = str(uuid4())

try:
event = RocketLaunchedEvent(
event_id=event_id,
trace_id=trace_id,
customer_id=schema.customer_id,
launched_at=...
)

await reporting_event_publisher.publish(event)

except Exception as exception:
# registrar a ocorrência para recuperação: event_id, etc.
pass

return launch_result

```

A assinatura acima mostra a nova dependência seguindo o padrão de Dependency Injection já utilizado pelo projeto. O `ReportingEventPublisher` é um componente novo da solução e deve ser registrado no Container. Sua implementação concreta conhece o mecanismo de mensageria; o endpoint conhece apenas a abstração de publicação.

6. RocketLaunchedEvent

O evento representa o fato de que o lançamento foi concluído. A criação do objeto ocorre depois de `service.launch()` retornar com sucesso. Uma estrutura mínima contém:

```

event_id – identificador único do evento
trace_id – identificador de rastreamento já existente no endpoint
customer_id – cliente do lançamento
launched_at – instante associado ao lançamento concluído

```

O `event_id` acompanha a ocorrência durante o workflow e pode ser usado tanto para diagnóstico quanto para idempotência no Reporting DB.

7. ReportingEventPublisher e SQS

O `ReportingEventPublisher` encapsula a publicação do evento. Dessa forma, o endpoint não precisa conhecer detalhes de SDK, credenciais, configuração de fila ou chamadas específicas do SQS.

A fila SQS funciona como a fronteira de desacoplamento. Depois que uma mensagem foi publicada, a indisponibilidade do Reporting DB não precisa afetar o serviço de lançamento: o consumidor pode aguardar a recuperação da dependência e tentar novamente.

A escolha de SQS é adequada ao cenário apresentado porque o requisito descreve um workflow de reporting único interessado nos dados. Kafka também poderia ser utilizado, mas introduziria uma infraestrutura mais ampla do que a necessidade explicitamente apresentada.

8. Reporting Consumer

O consumidor recebe a mensagem da fila, processa o evento e persiste o registro no Reporting DB. A confirmação (ACK/delete) deve ocorrer somente depois da persistência bem-sucedida.

Falhas transitórias de persistência devem utilizar retry com backoff. Após um limite de tentativas, uma Dead Letter Queue (DLQ) pode receber as mensagens que não puderam ser processadas, permitindo análise e recuperação sem manter o processamento indefinidamente.

9. Idempotência

Mensagerias como SQS podem resultar em entrega repetida. Para evitar que um mesmo lançamento seja contado mais de uma vez, o `event_id` deve ser tratado como identificador idempotente. No Reporting DB, ele pode possuir uma restrição UNIQUE e o consumidor pode utilizar INSERT/UPSERT conforme a estratégia adotada.

Assim, uma segunda entrega do mesmo `RocketLaunchedEvent` não gera uma segunda ocorrência lógica no relatório.

10. Reporting DB e relatório de 30 dias

O Reporting DB é uma base paralela, destinada exclusivamente às necessidades do reporting. O relatório solicitado pode considerar os lançamentos ocorridos nos últimos 30 dias e agrupá-los por cliente.

```
SELECT
customer_id,
COUNT(*) AS total_launches
FROM rocket_launch_report
WHERE launched_at >= NOW() - INTERVAL '30 days'
GROUP BY customer_id;
```

O enunciado não especifica o canal de entrega do relatório. Portanto, a proposta não assume se ele será disponibilizado por endpoint, arquivo, dashboard ou outro meio. Essa decisão pode ser adicionada posteriormente sem alterar o mecanismo de ingestão.

11. Tratamento de falhas e recuperação

A publicação para reporting é uma operação secundária. Se ela falhar após o lançamento ter sido concluído, a exceção não deve ser propagada como erro do lançamento. O endpoint mantém o `launch_result` e retorna o resultado da operação principal.

Após tentativas limitadas, a falha deve ser registrada com informações suficientes para diagnóstico e recuperação operacional, como `event_id`, `trace_id`, `customer_id`, timestamp e dados da exceção. Em uma situação excepcional, esses dados permitem localizar a ocorrência e realizar reprocessamento ou correção operacional.

Essa decisão é deliberada: priorizamos a disponibilidade e a continuidade do serviço principal, conforme exigido pelo enunciado, em vez de fazer o lançamento depender do workflow de reporting.

12. Limite da garantia de entrega

SQS garante a durabilidade da mensagem depois que ela foi efetivamente publicada. Ele não elimina, por si só, a pequena janela entre a conclusão do lançamento e uma eventual falha antes da publicação.

A solução proposta não tenta resolver essa janela tornando o Reporting DB ou o SQS uma dependência transacional do lançamento, pois isso contrariaria a premissa de isolamento do workflow secundário. Em vez disso, uma falha persistente de publicação é tratada como uma ocorrência recuperável e observável.

13. Por que não Kafka?

Kafka resolveria o desacoplamento e ofereceria recursos adicionais de retenção e replay. Porém, o requisito apresentado não indica múltiplos consumidores independentes, necessidade de replay por vários sistemas ou uma arquitetura de streaming. Para um único workflow de reporting, SQS é suficiente e mantém a solução mais simples.

14. Resumo

A proposta mantém o endpoint de lançamento como fluxo crítico, reutiliza a solicitação existente, cria um `RocketLaunchedEvent` após o sucesso do lançamento, publica esse evento por meio de um `ReportingEventPublisher` em uma fila SQS e processa a mensagem de forma independente no Reporting Consumer.

O Reporting DB é paralelo ao banco principal e serve de projeção para o relatório de 30 dias. Retry, backoff, idempotência e DLQ protegem o workflow secundário. Se o reporting falhar, o lançamento já concluído continua sendo considerado bem-sucedido.